# Detecting Spam Messages with Natural Language Processing

## Preprocessing, text encoding and supervised classifiers evaluated from small to large message collections

Project report | DLBAIPNLP01 - Project: NLP | Task 2: Text Classification for Spam Detection

Yousef Mohamed Mahmoud Mahmoud Alashmawy
9 October 2026

Code repository: https://github.com/yousefalashmawi/spam-detection-nlp

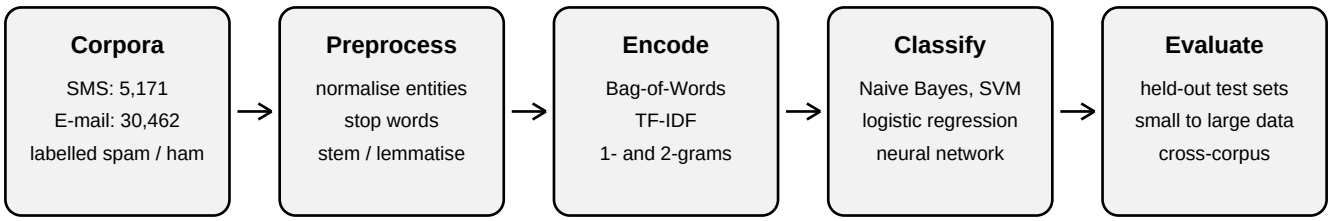

### Abstract

Spam messages are unsolicited bulk messages that waste attention and can carry phishing links or malware. This project develops a natural language processing (NLP) system that classifies text messages and e-mails as spam or legitimate (ham). Two public corpora were used: the SMS Spam Collection with 5,171 unique messages as the small scenario and Enron-Spam with 30,462 unique e-mails as the large scenario. Messages were normalised, stop words were removed, words were stemmed or lemmatised, and the text was encoded as Bag-of-Words or TF-IDF n-gram vectors. Naive Bayes, a linear support vector machine (SVM), logistic regression and a small feed-forward neural network were trained and evaluated on held-out test data, with learning curves from 100 training messages up to the full corpora and tests across the two message types.

Replacing numbers, phone numbers, URLs and currency symbols with placeholder tokens was the most useful preprocessing step for SMS, raising F1 by 2.8 points for the linear models, while preprocessing hardly changed e-mail results. Linear models matched or beat the neural network at a fraction of its training time. Models trained on one message type failed on the other, so the final system routes each message to a channel-specific model. On untouched test data the SMS model reached 98.94% accuracy and flagged 2 of 904 legitimate messages; the e-mail model reached 99.20% accuracy with an F1 score of 0.992.


### Graphical abstract

**Fig. 1 Graphical abstract**




Result: channel-specific models. SMS accuracy 98.94%, e-mail accuracy 99.20%; legitimate messages flagged: 0.22% (SMS), 1.04% (e-mail).

Source: Own representation of the system developed in this project. Figures in the bottom line are measured on the held-out test sets (see Section 4).

# 1. Introduction

Text classification assigns a document to one of a set of predefined categories; spam detection is its classic binary case, in which every incoming message is labelled as spam or ham (Jurafsky & Martin, 2013). A filter is only useful if it removes most spam while almost never hiding a legitimate message, because a lost personal or business message costs the user far more than one unwanted advertisement. This project builds such a filter and examines each step of the NLP pipeline in different scenarios. It addresses four questions:

**Q1** Which preprocessing steps (normalisation, stop-word removal, stemming, lemmatising) and which text encoding (Bag-of-Words or TF-IDF) improve classification?
**Q2** How does performance develop from a small to a large training set?
**Q3** Do probabilistic, discriminative and neural classifiers differ in accuracy and cost?
**Q4** Does a model trained on one kind of message generalise to another kind?

## 1.1 Related work

Naive Bayes classifiers have long been a standard approach to spam filtering. Metsis et al. (2006) compared several Naive Bayes variants on e-mail and, for that purpose, assembled the Enron-Spam corpus, which combines legitimate mail from Enron employees with collected spam. For mobile messages, Almeida et al. (2011) published the SMS Spam Collection and reported results of established classifiers as baselines, noting that short, abbreviation-rich SMS texts are harder to filter than e-mails. More recent work uses deep learning: Elakkiya et al. (2021) combine textual features with an attention mechanism to detect spam in social media, and Huan et al. (2022) join convolutional and bidirectional long short-term memory layers so that a text classifier captures both local n-gram patterns and longer-range word order. Such models are more expressive but need more data and computation. This project therefore compares classical linear models on sparse n-gram features with a neural network on the same features, and asks whether the extra cost is justified for this task.

## 1.2 Data collection

The task suggests the UCI Spambase data set. Spambase, however, contains only 57 pre-computed numeric features such as word and character frequencies for 4,601 e-mails, not the e-mail text (Hopkins et al., 1999). Stop-word removal, stemming and text encoding, which the task asks to evaluate, cannot be applied to it. Two public corpora with raw text and labels were therefore used (Tab. 1). The SMS Spam Collection (Almeida & Hidalgo, 2011) provides the small scenario. Enron-Spam (Metsis et al., 2006), in the CSV version prepared by Wiechmann (n.d.) with subject and body per e-mail, provides the large scenario. Exact duplicate texts were removed before any split; otherwise the same message could appear in both training and test data and inflate the scores.

**Tab. 1 Data sets after de-duplication**

| Corpus | Type | Raw | Unique | Spam share | Median length | Train / test |
|---|---|---|---|---|---|---|
| SMS Spam Collection | SMS | 5,574 | 5,171 | 12.6% | 12 words | 4,136 / 1,035 |
| Enron-Spam | E-mail | 33,665 | 30,462 | 47.8% | 154 words | 24,369 / 6,093 |

Source: Own representation of the loaded data; data from Almeida and Hidalgo (2011) and Metsis et al. (2006) via Wiechmann (n.d.). Raw counts exclude e-mails without any text.

The two corpora differ strongly. SMS spam is a minority class (about one message in eight) and SMS texts are short and informal, whereas Enron-Spam is almost balanced and its e-mails are more than ten times longer. This difference is used deliberately in Q4.

# 2. Method

## 2.1 Preprocessing

Five cumulative preprocessing levels were implemented (Tab. 2) so that the effect of each step can be measured separately. Stop words come from the NLTK English list (Bird et al., 2009). Stemming uses the Porter algorithm, which strips suffixes by rules (Porter, 1980); lemmatising maps words to dictionary base forms with WordNet (Miller, 1995), first as nouns and then as verbs. The placeholder tokens of the clean level are not stemmed.

**Tab. 2 Preprocessing levels**

| Level | Operation | Example output for a spam SMS |
|---|---|---|
| raw | Lower-casing only (baseline) | free entry in 2 a wkly comp to win fa cup final tkts 21st may ... |
| clean | URLs, e-mail addresses, phone numbers, currency symbols and numbers replaced by tokens; punctuation removed | free entry in tok_num a wkly comp to win fa cup final tkts st may ... |
| stopwords | clean + NLTK English stop words removed | free entry tok_num wkly comp win fa cup final tkts st may ... |
| stem | stopwords + Porter stemming | free entri tok_num wkli comp win fa cup final tkt st may ... |
| lemma | stopwords + WordNet lemmatising | free entry tok_num wkly comp win fa cup final tkts st may ... |

Source: Own representation of the implemented preprocessing (src/preprocess.py); example message from the SMS Spam Collection (Almeida & Hidalgo, 2011).

## 2.2 Encoding and classifiers

Preprocessed text was converted into sparse vectors of unigrams and bigrams that occur in at least two training messages, limited to 50,000 features. Two weightings were compared: raw counts (Bag-of-Words) and TF-IDF with sublinear term frequency, which down-weights terms that occur in many documents (Manning et al., 2008). The encoders were fitted on training data only. Four supervised classifiers from scikit-learn (Pedregosa et al., 2011) were trained on these vectors (Tab. 3).

**Tab. 3 Classifier settings**

| Classifier | Type | Settings |
|---|---|---|
| Multinomial Naive Bayes | Probabilistic, generative | Additive smoothing alpha = 0.1 |
| Linear SVM | Discriminative, maximum margin | C = 1, balanced class weights |
| Logistic regression | Discriminative, probabilistic | C = 10, liblinear solver, balanced class weights |
| MLP (neural network) | Feed-forward, deep learning baseline | One hidden layer of 64 ReLU units, Adam, early stopping, at most 50 epochs, 20,000 input features |

Source: Own representation of the implementation (src/models.py), based on Pedregosa et al. (2011).

## 2.3 Evaluation protocol

Each corpus was split once into a stratified 80% training and 20% test part (random seed 42). The test parts were never used for training or for choosing a configuration. Spam is the positive class. Reported metrics are accuracy, precision (share of flagged messages that are spam), recall (share of spam that is caught), F1 (harmonic mean of precision and recall), ROC-AUC and the false-positive rate, the share of legitimate messages wrongly flagged. Accuracy alone is misleading on the imbalanced SMS data: a filter that lets every message through is already about 87% accurate.

Four experiments answer Q1 to Q4. **E1** trains every combination of preprocessing level, encoding and classical classifier on each training set. **E2** trains on stratified random subsets from 100 messages to the full training set, five different subsets per size, and always tests on the same full test set. **E3** compares all four classifiers with 5-fold cross-validation on the training data and on the test set. **E4** trains the best

classifier on SMS only, e-mail only and both combined and tests it on both test sets. Finally, for each channel the preprocessing level and classifier were selected by cross-validated F1 on the training data, refitted and evaluated once on the test set. The experiments ran in Python 3.13 with scikit-learn 1.9 and NLTK 3.10 on a Linux machine with two CPU cores; the executed notebook in the appendix shows all code and outputs.

# 3. Results

## 3.1 Effect of preprocessing and encoding (E1)

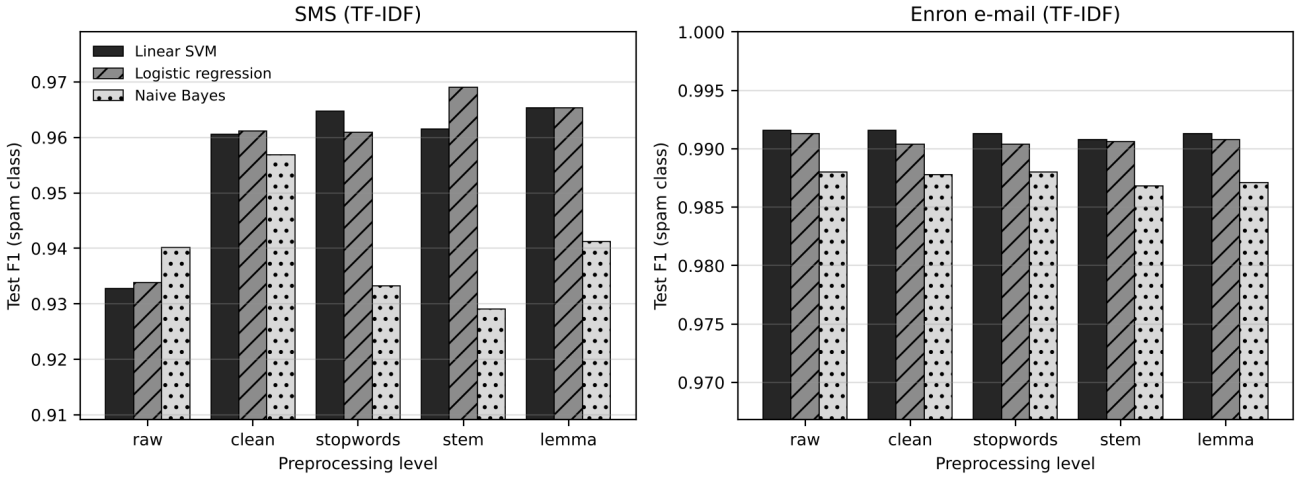
**Tab. 4 Test F1 by preprocessing level (TF-IDF encoding)**

| Corpus | Level | Naive Bayes | Linear SVM | Logistic regression |
|---|---|---|---|---|
| SMS | raw | 0.9402 | 0.9328 | 0.9339 |
| SMS | clean | 0.9569 | 0.9606 | 0.9612 |
| SMS | stopwords | 0.9333 | 0.9647 | 0.9609 |
| SMS | stem | 0.9291 | 0.9615 | 0.9690 |
| SMS | lemma | 0.9412 | 0.9653 | 0.9653 |
| E-mail | raw | 0.9880 | 0.9916 | 0.9913 |
| E-mail | clean | 0.9878 | 0.9916 | 0.9904 |
| E-mail | stopwords | 0.9880 | 0.9913 | 0.9904 |
| E-mail | stem | 0.9868 | 0.9908 | 0.9906 |
| E-mail | lemma | 0.9871 | 0.9913 | 0.9908 |

Source: Own representation of measured experiment outputs (results/e1_preprocessing.csv). Bag-of-Words results are in the same file.

For SMS, normalisation was the decisive step: replacing numbers, phone numbers, currency symbols and URLs by tokens raised F1 from 0.9328 to 0.9606 for the SVM and by 2.8 points on average for the two linear models (Tab. 4, Fig. 2). SMS spam typically asks the reader to call a premium number, text a short code or claim a cash prize; before normalisation, every number is a different rare word, afterwards the shared pattern becomes one strong feature. Stop-word removal, stemming and lemmatising changed the linear models by at most about one point, in both directions. They harmed Naive Bayes: removing stop words lowered its F1 by 2.4 points. In very short texts, function words such as 'you', 'your' or 'to' carry part of the persuasive style of spam, and Naive Bayes, which weighs every word independently, loses more from removing them. On e-mail, all levels lie within 0.1 points of each other; long texts contain enough evidence for the classifier to ignore noise. TF-IDF outperformed raw counts on average (SMS 0.952 vs. 0.950; e-mail 0.990 vs. 0.983).

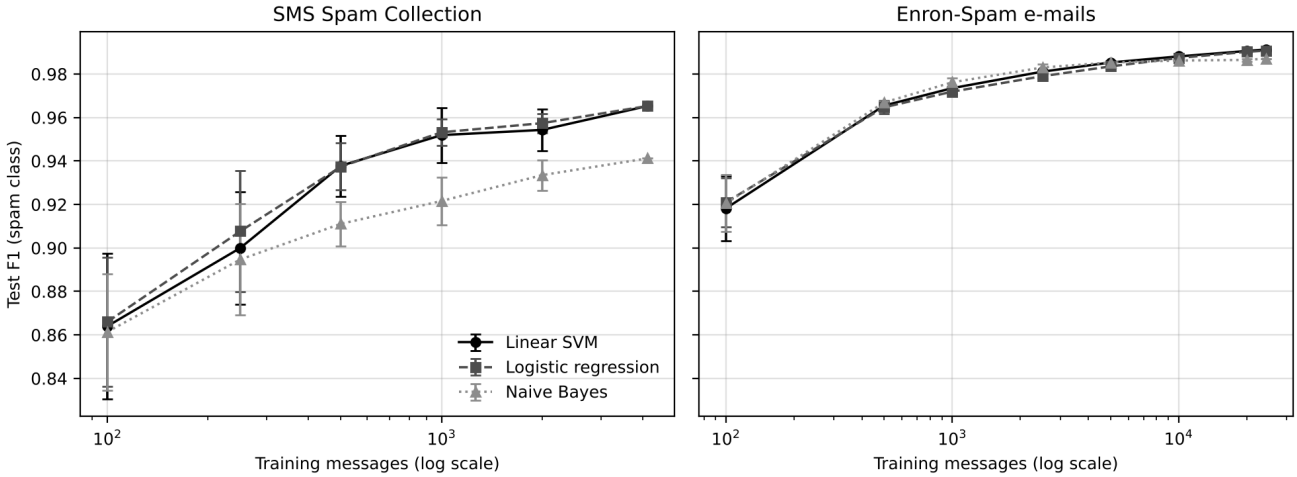
**Fig. 2 Test F1 by preprocessing level**




Source: Own representation of measured experiment outputs. Note the different vertical scales.

## 3.2 From small to large training sets (E2)

**Fig. 3 Learning curves**




Source: Own representation of measured experiment outputs (results/e2_learning_curve.csv). Points are means over five random training subsets, bars one standard deviation; the full training set is a single run.

Performance rose steeply up to about 1,000 training messages and then levelled off (Fig. 3). With 100 messages the SVM reached an F1 of 0.864 on SMS and 0.918 on e-mail; with the full training sets (4,136 and 24,369 messages) it reached 0.965 and 0.991. The spread between random subsets shrank from about 3 F1 points at 100 SMS messages to about 1 point at 2,000, and on e-mail to 0.2 points from 500 messages onwards, so small training sets give not only lower but also less reliable results. The false-positive rate on e-mail fell from 7.8% to 1.1%. Naive Bayes was competitive with little data but improved least as data grew, which matches the usual behaviour of a generative model with strong independence assumptions compared with discriminative models. The SMS curves are still rising at the full corpus size, so more labelled SMS data would probably help; the e-mail curves have almost converged.

## 3.3 Comparison of classifiers (E3)

**Tab. 5 Classifier comparison (lemmatised text, TF-IDF)**

| Corpus | Classifier | CV F1 (mean ± sd) | Accuracy | Precision | Recall | F1 | AUC | Train time |
|---|---|---|---|---|---|---|---|---|
| SMS | Naive Bayes | 0.940 ± 0.018 | 0.9855 | 0.9677 | 0.9160 | 0.9412 | 0.9937 | 0.2 s |
| SMS | Linear SVM | 0.945 ± 0.017 | 0.9913 | 0.9766 | 0.9542 | 0.9653 | 0.9978 | 0.2 s |
| SMS | Logistic regression | 0.946 ± 0.017 | 0.9913 | 0.9766 | 0.9542 | 0.9653 | 0.9975 | 0.1 s |
| SMS | MLP | 0.922 ± 0.025 | 0.9874 | 0.9917 | 0.9084 | 0.9482 | 0.9944 | 4.6 s |
| E-mail | Naive Bayes | 0.987 ± 0.001 | 0.9877 | 0.9870 | 0.9873 | 0.9871 | 0.9989 | 16.0 s |
| E-mail | Linear SVM | 0.990 ± 0.001 | 0.9916 | 0.9877 | 0.9948 | 0.9913 | 0.9992 | 16.2 s |
| E-mail | Logistic regression | 0.989 ± 0.002 | 0.9911 | 0.9870 | 0.9945 | 0.9908 | 0.9992 | 16.2 s |
| E-mail | MLP | 0.989 ± 0.001 | 0.9902 | 0.9884 | 0.9911 | 0.9897 | 0.9992 | 117.4 s |

Source: Own representation of measured experiment outputs (results/e3_models.csv). Training time is for one fit on the full training set.

Cross-validated and test scores agree closely, so the results do not depend on a lucky split (Tab. 5). The linear SVM and logistic regression were the strongest models; across both corpora the SVM had the highest mean cross-validated F1. The neural network did not improve on them: on SMS its F1 was 0.948 against 0.965 for the SVM, with the lowest and most variable cross-validation score, a sign of overfitting on only 4,136 messages. On e-mail it reached 0.990 but needed 7 times as long to train. With thousands of sparse n-gram features, spam and ham are already almost linearly separable, so a hidden layer adds cost without adding accuracy. All models achieved a ROC-AUC above 0.99, meaning their ranking of messages is very good and differences lie mainly in where the decision threshold falls (AUC = area under the ROC curve).

## 3.4 Generalisation across message types (E4)

**Tab. 6 Cross-corpus evaluation (Linear SVM, lemmatised text, TF-IDF)**

| Trained on | Tested on | Accuracy | Precision | Recall | F1 | Ham flagged |
|---|---|---|---|---|---|---|
| SMS | SMS | 0.9913 | 0.9766 | 0.9542 | 0.9653 | 0.33% |
| SMS | E-mail | 0.5444 | 0.5277 | 0.4414 | 0.4807 | 36.14% |
| E-mail | SMS | 0.3140 | 0.1448 | 0.9008 | 0.2495 | 77.10% |
| E-mail | E-mail | 0.9916 | 0.9877 | 0.9948 | 0.9913 | 1.13% |
| SMS + e-mail | SMS | 0.9585 | 0.7750 | 0.9466 | 0.8522 | 3.98% |
| SMS + e-mail | E-mail | 0.9839 | 0.9920 | 0.9742 | 0.9830 | 0.72% |

Source: Own representation of measured experiment outputs (results/e4_generalisation.csv).

A model is only reliable on the kind of text it was trained on (Tab. 6). The e-mail model flagged 77% of legitimate SMS as spam, because casual text messages look nothing like Enron business mail, and the SMS model detected fewer than half of the spam e-mails. Training on both corpora together largely solved this, but SMS performance still dropped to an F1 of 0.852, and 4.0% of legitimate SMS were flagged, compared with 0.3% for the SMS-only model. The six times larger and almost balanced e-mail corpus dominates the joint model. The final system therefore uses one model per channel.

# 4. Final system

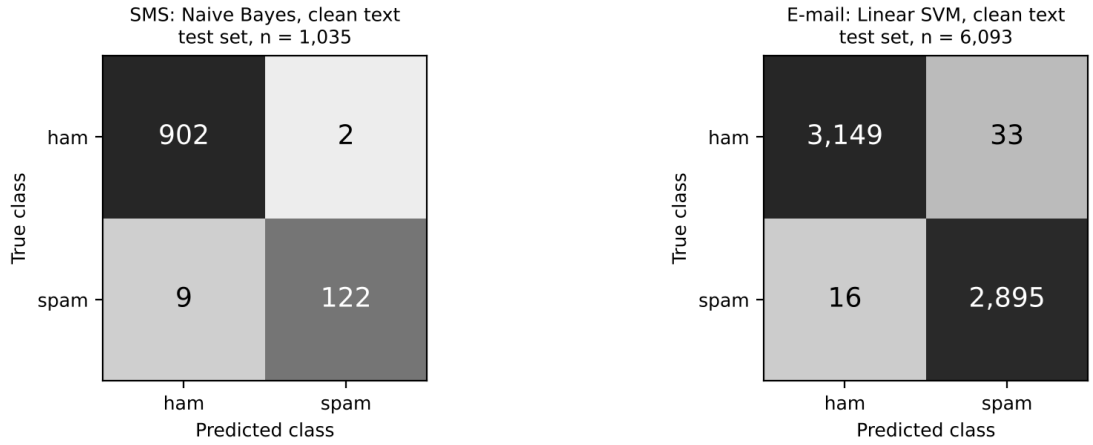
For each channel, all combinations of four preprocessing levels and three classical classifiers were scored by cross-validated F1 on the training data only (5 folds for SMS, 3 for e-mail). The best combination was refitted on the full training set and saved as a pipeline that accepts raw text. A simple router sends multi-line texts or texts longer than 60 words to the e-mail model and all others to the SMS model. The command-line tool *predict.py* classifies new messages with these pipelines. Tab. 7 reports their single evaluation on the untouched test sets.

**Tab. 7 Final models on the held-out test sets**

| | SMS model | E-mail model |
|---|---|---|
| **Selected by cross-validation** | Naive Bayes, clean text | Linear SVM, clean text |
| **Cross-validated F1 (training data)** | 0.960 | 0.991 |
| **Test messages** | 1,035 | 6,093 |
| **Accuracy** | 98.94% | 99.20% |
| **Precision / recall** | 98.39% / 93.13% | 98.87% / 99.45% |
| **F1 / ROC-AUC** | 0.957 / 0.996 | 0.992 / 0.999 |
| **Legitimate messages flagged** | 2 of 904 (0.22%) | 33 of 3,182 (1.04%) |
| **Spam missed** | 9 of 131 | 16 of 2,911 |
| **Training / prediction time** | 0.4 s / 0.05 ms per message | 30.4 s / 0.86 ms per message |

Source: Own representation of measured experiment outputs (results/final_model.json).

**Fig. 4 Confusion matrices of the final models**




Source: Own representation of measured experiment outputs; rows are true classes, columns predicted classes.

For SMS, cross-validation selected Naive Bayes on lightly cleaned text, without stop-word removal or lemmatising, in line with E1. It flagged only 2 legitimate messages while missing 9 spam messages (Fig. 4). This is the preferable trade-off for a spam filter: a missed spam message is a nuisance, a hidden legitimate one can be costly. Several missed spam messages imitate harmless content, such as a joke ("Latest News! Police station toilet stolen, cops have nothing to go on!") or a notification ("Missed call alert. These numbers called but left no message. 07008009200"). The flagged legitimate messages use product-like wording: "Nokia phone is lovly.." and "Unlimited texts. Limited minutes.". For e-mail, the linear SVM reached an F1 of 0.992. Its strongest spam indicators include your, http, remove, no, software, here, while ham indicators include enron, vince, louise, thanks, attached, energy; part of the signal is thus specific to Enron (Section 5).

## 5. Discussion and limitations

The experiments show that the value of each pipeline step depends on the kind of text. For short, noisy SMS, normalisation of numbers, currencies and links mattered more than any linguistic processing, and stemming or lemmatising brought no consistent gain. Their main advantage, merging word forms, is small when bigrams and thousands of training examples are available, and they remove stylistic cues such as abbreviations. For long e-mails the choice of preprocessing hardly mattered. Data size mattered more than model choice: going from 100 to 1,000 training messages gained more F1 than any change of classifier at full size. Among the classifiers, discriminative linear models were the best compromise of accuracy, speed and transparency, because their weights directly show which n-grams drive a decision.

The most important finding for deployment is the lack of transfer between message types (E4). A filter is only as good as the match between its training data and the messages it will see, so the system keeps one model per channel and should be retrained on data from its actual environment.

The study has limitations. Enron-Spam consists of e-mails from the early 2000s, and its ham comes from one company; several strong ham indicators are Enron-specific names and terms, so the e-mail scores are likely optimistic for a general mailbox. Modern spam relies more on phishing links, images and several languages. Each corpus was split only once, although cross-validation and repeated subsamples in E2 reduce this risk. The decision threshold was left at the default; in practice it could be tuned to a target false-positive rate. Finally, transformer-based language models were not tested; given the near-perfect ROC-AUC of the linear models, their expected benefit here is small compared with their computational cost, but they could help with more varied data.

## 6. Conclusion

This project developed and evaluated an NLP system for spam detection. It collects two public labelled corpora, preprocesses the text with normalisation, stop-word removal, stemming or lemmatising, encodes it as Bag-of-Words or TF-IDF n-grams, trains Naive Bayes, a linear SVM, logistic regression and a neural network, and evaluates them from small to large data sets and across message types. Normalising entities was the most useful preprocessing step for SMS (Q1), performance improved quickly up to about 1,000 training messages and then levelled off (Q2), linear models were as accurate as the neural network at much lower cost (Q3), and models did not transfer between SMS and e-mail (Q4). The final channel-specific system classifies new messages with 98.94% (SMS) and 99.20% (e-mail) accuracy on held-out test data while flagging very few legitimate messages. Future work should retrain and test the system on recent, multilingual messages, tune the decision threshold, and compare it with transformer-based models.

# References


Almeida, T. A., Gómez Hidalgo, J. M., & Yamakami, A. (2011). Contributions to the study of SMS spam filtering: New collection and results. In *Proceedings of the 11th ACM Symposium on Document Engineering* (pp. 259–262). Association for Computing Machinery. https://doi.org/10.1145/2034691.2034742

Almeida, T. A., & Hidalgo, J. M. G. (2011). *SMS spam collection* [Data set]. UCI Machine Learning Repository. https://doi.org/10.24432/C5CC84

Bird, S., Klein, E., & Loper, E. (2009). *Natural language processing with Python*. O'Reilly Media.

Elakkiya, E., Selvakumar, S., & Leela Velusamy, R. (2021). TextSpamDetector: Textual content based deep learning framework for social spam detection using conjoint attention mechanism. *Journal of Ambient Intelligence and Humanized Computing, 12*(10), 9287–9302. https://doi.org/10.1007/s12652-020-02640-5

Hopkins, M., Reeber, E., Forman, G., & Suermondt, J. (1999). *Spambase* [Data set]. UCI Machine Learning Repository. https://doi.org/10.24432/C53G6X

Huan, H., Guo, Z., Cai, T., & He, Z. (2022). A text classification method based on a convolutional and bidirectional long short-term memory model. *Connection Science, 34*(1), 2108–2124. https://doi.org/10.1080/09540091.2022.2098926

Jurafsky, D., & Martin, J. H. (2013). *Speech and language processing: An introduction to natural language processing, computational linguistics, and speech recognition* (2nd ed.). Pearson Prentice Hall.

Manning, C. D., Raghavan, P., & Schütze, H. (2008). *Introduction to information retrieval*. Cambridge University Press.

Metsis, V., Androutsopoulos, I., & Paliouras, G. (2006). Spam filtering with naive Bayes – Which naive Bayes? In *Proceedings of the Third Conference on Email and Anti-Spam (CEAS 2006)*. Mountain View, CA, United States.

Miller, G. A. (1995). WordNet: A lexical database for English. *Communications of the ACM, 38*(11), 39–41. https://doi.org/10.1145/219717.219748

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research, 12*, 2825–2830.

Porter, M. F. (1980). An algorithm for suffix stripping. *Program, 14*(3), 130–137. https://doi.org/10.1108/eb046814

Wiechmann, M. (n.d.). *enron_spam_data* [Data set and source code]. GitHub. https://github.com/MWiechmann/enron_spam_data


## Code and appendix

The complete source code, tests, results and trained models are available at https://github.com/yousefalashmawi/spam-detection-nlp. The appendix that follows is a PDF rendering of the executed notebook *Spam_Detection_Experiments.ipynb*, which contains every code cell and its observed output; the same notebook is part of the repository.

# Appendix: Executed experiment notebook

Spam_Detection_Experiments.ipynb | code and observed outputs

## Spam detection with NLP - executed experiment notebook

DLBAIPNLP01 - Project: NLP, Task 2 (Text Classification for Spam Detection). The notebook runs every step of the system: data collection, preprocessing, encoding, training, evaluation from small to large data sets, cross-corpus generalisation, final model selection and classification of new messages. All functions are imported from the src/ package of the repository.

### In [1] - notebook cell 2

```
import sys, platform, time, json, warnings
from pathlib import Path
ROOT = Path.cwd().parent if Path.cwd().name == "notebooks" else Path.cwd()
sys.path.insert(0, str(ROOT / "src"))
warnings.filterwarnings("ignore", category=UserWarning)

import numpy as np, pandas as pd, sklearn, nltk, matplotlib
from IPython.display import Image, display
pd.set_option("display.width", 120); pd.set_option("display.max_colwidth", 70)

import data, preprocess, models, experiments
print("Python", platform.python_version(), "| scikit-learn", sklearn.__version__,
      "| NLTK", nltk.__version__, "| pandas", pd.__version__, "| NumPy", np.__version__)
print("Machine:", platform.machine(), platform.system())
```

Observed output

```
Python 3.13.16 | scikit-learn 1.9.1 | NLTK 3.10.3 | pandas 3.0.5 | NumPy 2.5.3
Machine: x86_64 Linux
```

## 1. Data collection

Two public labelled corpora are downloaded automatically: the SMS Spam Collection (small scenario) and Enron-Spam (large scenario). Exact duplicate texts are removed before splitting so that no message can appear in both the training and the test set.

### In [2] - notebook cell 4

```
sms_raw, enron_raw = data.load_sms(dedupe=False), data.load_enron(dedupe=False)
sms, enron = data.load_sms(), data.load_enron()
stats = pd.DataFrame({"SMS": data.describe(sms), "Enron e-mail": data.describe(enron)}).T
stats.insert(0, "raw messages", [len(sms_raw), len(enron_raw)])
stats.insert(1, "duplicates removed", [len(sms_raw) - len(sms), len(enron_raw) - len(enron)])
print(stats.to_string())
```

Observed output

```
              raw messages  duplicates removed  messages     spam      ham  spam_share  median_tokens
    mean_tokens
SMS                   5574                 403    5171.0    653.0   4518.0      0.1263           12.0
             15.4
Enron e-mail         33665                3203   30462.0  14552.0  15910.0      0.4777          154.0
            306.0
```

### In [3] - notebook cell 5

```
print("Example SMS messages:")
print(sms.groupby("label").head(3)[["label", "text"]].to_string(index=False))
```

Observed output

```
Example SMS messages:
 label
                                                  text
     0                                 Go until jurong point, crazy.. Available only in bugis
    n great world la e buffet... Cine there got amore wat...
     0
                             Ok lar... Joking wif u oni...
     1   Free entry in 2 a wkly comp to win FA Cup final tkts 21st May 2005. Text FA to 87121 to receive
    entry question(std txt rate)T&C's apply 08452810075over18's
     0
           U dun say so early hor... U c already then say...
     1           FreeMsg Hey there darling it's been 3 week's now and no word back! I'd like some fun you up
```

```
     for it still? Tb ok! XxX std chgs to send, £1.50 to rcv
     1 WINNER!! As a valued network customer you have been selected to receivea £900 prize reward! To claim
    call 09061701461. Claim code KL341. Valid 12 hours only.
```

**In [4] - notebook cell 6**

```
corpora = {"sms": experiments.split(sms), "enron": experiments.split(enron)}
for name, (tr, te) in corpora.items():
    print(f"{name:6s} train={len(tr):6d} (spam {tr.label.mean():.3f})  "
          f"test={len(te):5d} (spam {te.label.mean():.3f})")
```

Observed output

```
sms    train=  4136 (spam 0.126)  test= 1035 (spam 0.127)
enron  train= 24369 (spam 0.478)  test= 6093 (spam 0.478)
```

## 2. Preprocessing

Five cumulative preprocessing levels: raw (lower-case only), clean (placeholder tokens for URLs, e-mail addresses, phone numbers, currency and numbers; punctuation removed), stopwords (+ NLTK English stop-word removal), stem (+ Porter stemming) and lemma (+ WordNet lemmatising).

**In [5] - notebook cell 8**

```
example = sms[sms.label == 1].text.iloc[0]
print("Original :", example)
for mode in preprocess.MODES:
    print(f"{mode:9s}:", preprocess.preprocess(example, mode))
```

Observed output

```
Original : Free entry in 2 a wkly comp to win FA Cup final tkts 21st May 2005. Text FA to 87121 to receive
    entry question(std txt rate)T&C's apply 08452810075over18's
raw      : free entry in 2 a wkly comp to win fa cup final tkts 21st may 2005. text fa to 87121 to receive
    entry question(std txt rate)t&c's apply 08452810075over18's
clean    : free entry in tok_num a wkly comp to win fa cup final tkts st may tok_num text fa to tok_phone to
     receive entry question std txt rate t c s apply over s
stopwords: free entry tok_num wkly comp win fa cup final tkts st may tok_num text fa tok_phone receive entry
     question std txt rate apply
stem     : free entri tok_num wkli comp win fa cup final tkt st may tok_num text fa tok_phone receiv entri
    question std txt rate appli
lemma    : free entry tok_num wkly comp win fa cup final tkts st may tok_num text fa tok_phone receive entry
     question std txt rate apply
```

**In [6] - notebook cell 9**

```
# Preprocess every message once per level (cached for the experiments below).
import shutil
shutil.rmtree(experiments.CACHE, ignore_errors=True)
for mode in preprocess.MODES:
    t0 = time.perf_counter()
    for name, (tr, te) in corpora.items():
        experiments.cached_preprocess(pd.concat([tr, te]), name, mode)
    print(f"{mode:9s} preprocessed both corpora in {time.perf_counter() - t0:6.1f} s")
```

Observed output

```
raw       preprocessed both corpora in    2.9 s
clean     preprocessed both corpora in   10.4 s
stopwords preprocessed both corpora in   10.5 s
stem      preprocessed both corpora in   59.3 s
lemma     preprocessed both corpora in   36.6 s
```

**In [7] - notebook cell 10**

```
# Encoding: unigram + bigram TF-IDF (or Bag-of-Words counts), min_df = 2.
X_tr, y_tr, X_te, y_te = experiments.prepared(corpora, "lemma")["sms"]
vec = models.make_vectorizer("tfidf").fit(X_tr)
M = vec.transform(X_tr)
print("SMS vocabulary size:", len(vec.vocabulary_), "| training matrix:", M.shape,
      f"| non-zero share: {M.nnz / (M.shape[0] * M.shape[1]):.5f}")
```

Observed output

```
SMS vocabulary size: 5901 | training matrix: (4136, 5901) | non-zero share: 0.00174
```

## 3. Experiment E1 - effect of each preprocessing step

Every preprocessing level x encoding (BoW, TF-IDF) x classifier (Naive Bayes, linear SVM, logistic regression) is trained on the training split and scored on the fixed test split of each corpus.

### In [8] - notebook cell 12

```
classic = ["naive_bayes", "linear_svm", "logistic_regression"]
t0 = time.perf_counter()
e1 = experiments.e1_preprocessing(corpora, classic, quick=False)
e1.to_csv(experiments.RESULTS / "e1_preprocessing.csv", index=False)
experiments.plot_preprocessing(e1)
print(f"E1 finished in {time.perf_counter() - t0:.0f} s")
```

Observed output

```
  E1 sms   raw       bow   naive_bayes          F1=0.9412
  E1 sms   raw       bow   linear_svm           F1=0.9187
  E1 sms   raw       bow   logistic_regression  F1=0.9280
  E1 sms   raw       tfidf naive_bayes          F1=0.9402
  E1 sms   raw       tfidf linear_svm           F1=0.9328
  E1 sms   raw       tfidf logistic_regression  F1=0.9339
  E1 enron raw       bow   naive_bayes          F1=0.9833
  E1 enron raw       bow   linear_svm           F1=0.9808
  E1 enron raw       bow   logistic_regression  F1=0.9851
  E1 enron raw       tfidf naive_bayes          F1=0.9880
  E1 enron raw       tfidf linear_svm           F1=0.9916
  E1 enron raw       tfidf logistic_regression  F1=0.9913
  E1 sms   clean     bow   naive_bayes          F1=0.9615
  E1 sms   clean     bow   linear_svm           F1=0.9524
  E1 sms   clean     bow   logistic_regression  F1=0.9565
  E1 sms   clean     tfidf naive_bayes          F1=0.9569
  E1 sms   clean     tfidf linear_svm           F1=0.9606
  E1 sms   clean     tfidf logistic_regression  F1=0.9612
  E1 enron clean     bow   naive_bayes          F1=0.9819
  E1 enron clean     bow   linear_svm           F1=0.9821
  E1 enron clean     bow   logistic_regression  F1=0.9858
  E1 enron clean     tfidf naive_bayes          F1=0.9878
  E1 enron clean     tfidf linear_svm           F1=0.9916
  E1 enron clean     tfidf logistic_regression  F1=0.9904
  E1 sms   stopwords bow   naive_bayes          F1=0.9237
  E1 sms   stopwords bow   linear_svm           F1=0.9565
  E1 sms   stopwords bow   logistic_regression  F1=0.9565
  E1 sms   stopwords tfidf naive_bayes          F1=0.9333
  E1 sms   stopwords tfidf linear_svm           F1=0.9647
  E1 sms   stopwords tfidf logistic_regression  F1=0.9609
  E1 enron stopwords bow   naive_bayes          F1=0.9845
  E1 enron stopwords bow   linear_svm           F1=0.9806
  E1 enron stopwords bow   logistic_regression  F1=0.9853
  E1 enron stopwords tfidf naive_bayes          F1=0.9880
  E1 enron stopwords tfidf linear_svm           F1=0.9913
  E1 enron stopwords tfidf logistic_regression  F1=0.9904
  E1 sms   stem      bow   naive_bayes          F1=0.9323
  E1 sms   stem      bow   linear_svm           F1=0.9728
  E1 sms   stem      bow   logistic_regression  F1=0.9688
  E1 sms   stem      tfidf naive_bayes          F1=0.9291
  E1 sms   stem      tfidf linear_svm           F1=0.9615
  E1 sms   stem      tfidf logistic_regression  F1=0.9690
  E1 enron stem      bow   naive_bayes          F1=0.9842
  E1 enron stem      bow   linear_svm           F1=0.9777
  E1 enron stem      bow   logistic_regression  F1=0.9841
  E1 enron stem      tfidf naive_bayes          F1=0.9868
  E1 enron stem      tfidf linear_svm           F1=0.9908
  E1 enron stem      tfidf logistic_regression  F1=0.9906
  E1 sms   lemma     bow   naive_bayes          F1=0.9358
  E1 sms   lemma     bow   linear_svm           F1=0.9690
  E1 sms   lemma     bow   logistic_regression  F1=0.9688
  E1 sms   lemma     tfidf naive_bayes          F1=0.9412
  E1 sms   lemma     tfidf linear_svm           F1=0.9653
  E1 sms   lemma     tfidf logistic_regression  F1=0.9653
  E1 enron lemma     bow   naive_bayes          F1=0.9838
  E1 enron lemma     bow   linear_svm           F1=0.9798
  E1 enron lemma     bow   logistic_regression  F1=0.9846
  E1 enron lemma     tfidf naive_bayes          F1=0.9871
  E1 enron lemma     tfidf linear_svm           F1=0.9913
  E1 enron lemma     tfidf logistic_regression  F1=0.9908

E1 finished in 737 s
```

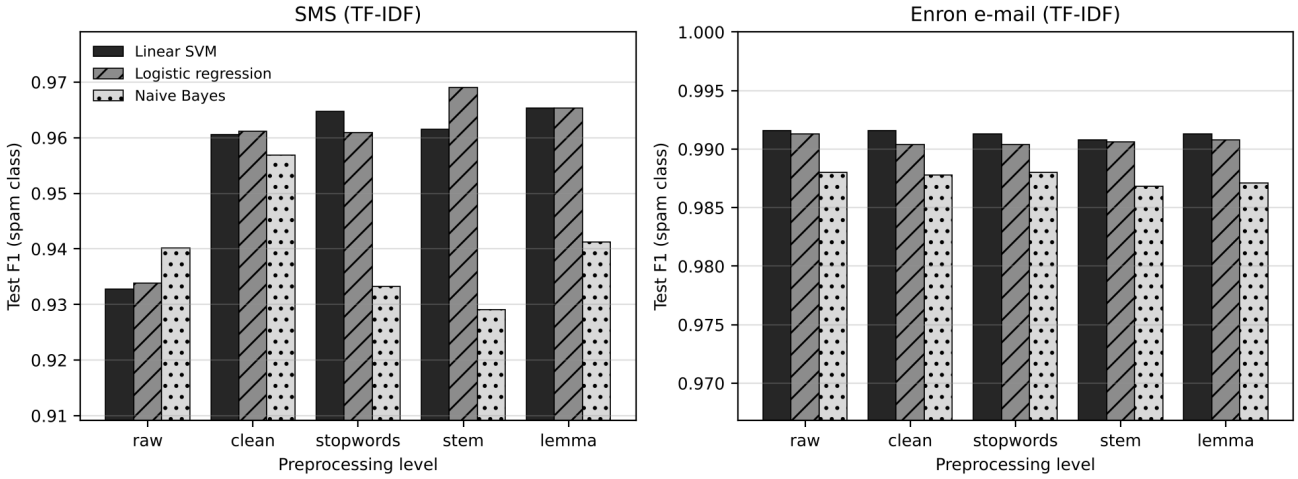
**In [9] - notebook cell 13**

```
print(e1.pivot_table(index=["corpus", "vectorizer", "preprocessing"], columns="model",
                     values="f1").round(4).to_string())
print()
print("Mean F1 by encoding:")
print(e1.groupby(["corpus", "vectorizer"]).f1.mean().round(4).to_string())
display(Image(str(experiments.FIGURES / "preprocessing_ablation.png"), width=800))
```

Observed output

```
model                           linear_svm  logistic_regression  naive_bayes
corpus vectorizer preprocessing
enron  bow        clean              0.9821               0.9858       0.9819
                  lemma              0.9798               0.9846       0.9838
                  raw                0.9808               0.9851       0.9833
                  stem               0.9777               0.9841       0.9842
                  stopwords          0.9806               0.9853       0.9845
       tfidf      clean              0.9916               0.9904       0.9878
                  lemma              0.9913               0.9908       0.9871
                  raw                0.9916               0.9913       0.9880
                  stem               0.9908               0.9906       0.9868
                  stopwords          0.9913               0.9904       0.9880
sms    bow        clean              0.9524               0.9565       0.9615
                  lemma              0.9690               0.9688       0.9358
                  raw                0.9187               0.9280       0.9412
                  stem               0.9728               0.9688       0.9323
                  stopwords          0.9565               0.9565       0.9237
       tfidf      clean              0.9606               0.9612       0.9569
                  lemma              0.9653               0.9653       0.9412
                  raw                0.9328               0.9339       0.9402
                  stem               0.9615               0.9690       0.9291
                  stopwords          0.9647               0.9609       0.9333

Mean F1 by encoding:
corpus  vectorizer
enron   bow           0.9829
        tfidf         0.9899
sms     bow           0.9495
        tfidf         0.9517
```




## 4. Experiment E2 - from small to large training sets

Models are trained on stratified random subsets of the training split (five different subsets per size) and always scored on the same full test split.

**In [10] - notebook cell 15**

```
t0 = time.perf_counter()
e2_runs, e2 = experiments.e2_learning_curve(corpora, classic, quick=False)
e2_runs.to_csv(experiments.RESULTS / "e2_learning_curve_runs.csv", index=False)
e2.to_csv(experiments.RESULTS / "e2_learning_curve.csv", index=False)
experiments.plot_learning_curves(e2)
print(f"E2 finished in {time.perf_counter() - t0:.0f} s")
```

Observed output

```
  E2 sms   n=   100 done
```

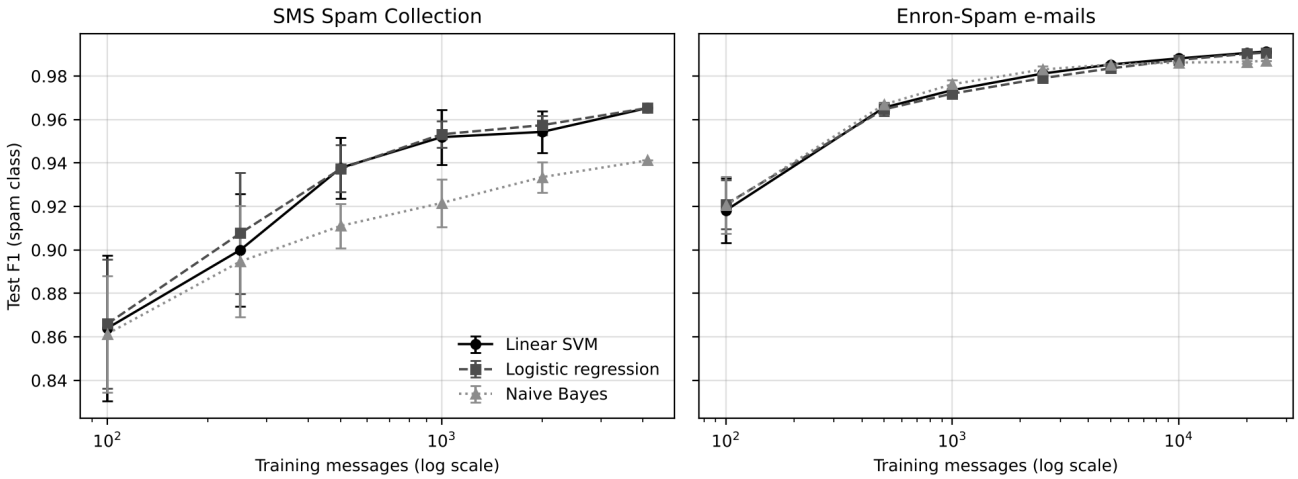
```
  E2 sms   n=   250 done
  E2 sms   n=   500 done
  E2 sms   n=  1000 done
  E2 sms   n=  2000 done
  E2 sms   n=  4136 done
  E2 enron n=   100 done
  E2 enron n=   500 done
  E2 enron n=  1000 done
  E2 enron n=  2500 done
  E2 enron n=  5000 done
  E2 enron n= 10000 done
  E2 enron n= 20000 done
  E2 enron n= 24369 done
E2 finished in 767 s
```

### In [11] - notebook cell 16

```
cols = ["corpus", "train_size", "model", "accuracy_mean", "f1_mean", "f1_std",
        "false_positive_rate_mean"]
print(e2[cols].to_string(index=False))
display(Image(str(experiments.FIGURES / "learning_curves.png"), width=800))
```

Observed output

```
corpus  train_size              model  accuracy_mean  f1_mean  f1_std  false_positive_rate_mean
 enron         100          linear_svm         0.9214   0.9182  0.0150                    0.0785
 enron         100 logistic_regression         0.9246   0.9210  0.0112                    0.0713
 enron         100         naive_bayes         0.9267   0.9205  0.0131                    0.0399
 enron         500          linear_svm         0.9665   0.9655  0.0019                    0.0475
 enron         500 logistic_regression         0.9657   0.9647  0.0030                    0.0491
 enron         500         naive_bayes         0.9687   0.9669  0.0007                    0.0195
 enron        1000          linear_svm         0.9743   0.9735  0.0006                    0.0366
 enron        1000 logistic_regression         0.9727   0.9719  0.0010                    0.0398
 enron        1000         naive_bayes         0.9775   0.9762  0.0019                    0.0157
 enron        2500          linear_svm         0.9817   0.9811  0.0006                    0.0267
 enron        2500 logistic_regression         0.9798   0.9790  0.0010                    0.0295
 enron        2500         naive_bayes         0.9838   0.9830  0.0017                    0.0136
 enron        5000          linear_svm         0.9859   0.9853  0.0007                    0.0201
 enron        5000 logistic_regression         0.9841   0.9835  0.0008                    0.0227
 enron        5000         naive_bayes         0.9860   0.9853  0.0005                    0.0132
 enron       10000          linear_svm         0.9886   0.9881  0.0013                    0.0156
 enron       10000 logistic_regression         0.9879   0.9874  0.0009                    0.0168
 enron       10000         naive_bayes         0.9869   0.9862  0.0010                    0.0125
 enron       20000          linear_svm         0.9910   0.9906  0.0004                    0.0118
 enron       20000 logistic_regression         0.9906   0.9902  0.0005                    0.0124
 enron       20000         naive_bayes         0.9871   0.9865  0.0006                    0.0127
 enron       24369          linear_svm         0.9916   0.9913     NaN                    0.0113
 enron       24369 logistic_regression         0.9911   0.9908     NaN                    0.0119
 enron       24369         naive_bayes         0.9877   0.9871     NaN                    0.0119
   sms         100          linear_svm         0.9656   0.8640  0.0335                    0.0190
   sms         100 logistic_regression         0.9660   0.8660  0.0296                    0.0195
   sms         100         naive_bayes         0.9652   0.8614  0.0268                    0.0186
   sms         250          linear_svm         0.9745   0.9000  0.0259                    0.0148
   sms         250 logistic_regression         0.9764   0.9077  0.0279                    0.0139
   sms         250         naive_bayes         0.9735   0.8947  0.0255                    0.0142
   sms         500          linear_svm         0.9847   0.9378  0.0140                    0.0044
   sms         500 logistic_regression         0.9845   0.9374  0.0108                    0.0053
   sms         500         naive_bayes         0.9778   0.9111  0.0102                    0.0109
   sms        1000          linear_svm         0.9880   0.9519  0.0126                    0.0042
   sms        1000 logistic_regression         0.9884   0.9532  0.0061                    0.0035
   sms        1000         naive_bayes         0.9805   0.9216  0.0109                    0.0088
   sms        2000          linear_svm         0.9886   0.9543  0.0096                    0.0042
   sms        2000 logistic_regression         0.9894   0.9574  0.0044                    0.0040
   sms        2000         naive_bayes         0.9835   0.9335  0.0070                    0.0057
   sms        4136          linear_svm         0.9913   0.9653     NaN                    0.0033
   sms        4136 logistic_regression         0.9913   0.9653     NaN                    0.0033
   sms        4136         naive_bayes         0.9855   0.9412     NaN                    0.0044
```




## 5. Experiment E3 - model comparison including a neural network

Lemmatised text and TF-IDF features; 5-fold stratified cross-validation on the training split plus one evaluation on the held-out test split. mlp is a feed-forward neural network with one hidden layer of 64 units.

**In [12] - notebook cell 18**

```
t0 = time.perf_counter()
e3 = experiments.e3_models(corpora, classic + ["mlp"])
e3.to_csv(experiments.RESULTS / "e3_models.csv", index=False)
print(f"E3 finished in {time.perf_counter() - t0:.0f} s")
print(e3[["corpus", "model", "cv_f1_mean", "cv_f1_std", "test_accuracy", "test_precision",
          "test_recall", "test_f1", "test_roc_auc", "test_train_seconds"]].to_string(index=False))
```

Observed output

```
  E3 sms   naive_bayes          cvF1=0.9402 testF1=0.9412
  E3 sms   linear_svm           cvF1=0.9448 testF1=0.9653
  E3 sms   logistic_regression  cvF1=0.9459 testF1=0.9653
  E3 sms   mlp                  cvF1=0.9220 testF1=0.9482
  E3 enron naive_bayes          cvF1=0.9873 testF1=0.9871
  E3 enron linear_svm           cvF1=0.9902 testF1=0.9913
  E3 enron logistic_regression  cvF1=0.9891 testF1=0.9908
  E3 enron mlp                  cvF1=0.9894 testF1=0.9897
E3 finished in 868 s
corpus              model  cv_f1_mean  cv_f1_std  test_accuracy  test_precision  test_recall  test_f1
    test_roc_auc  test_train_seconds
   sms         naive_bayes      0.9402     0.0182         0.9855          0.9677       0.9160   0.9412
            0.9937               0.17
   sms          linear_svm      0.9448     0.0171         0.9913          0.9766       0.9542   0.9653
            0.9978               0.19
   sms logistic_regression      0.9459     0.0174         0.9913          0.9766       0.9542   0.9653
            0.9975               0.15
   sms                 mlp      0.9220     0.0247         0.9874          0.9917       0.9084   0.9482
            0.9944               4.63
 enron         naive_bayes      0.9873     0.0014         0.9877          0.9870       0.9873   0.9871
            0.9989              15.98
 enron          linear_svm      0.9902     0.0008         0.9916          0.9877       0.9948   0.9913
            0.9992              16.23
 enron logistic_regression      0.9891     0.0015         0.9911          0.9870       0.9945   0.9908
            0.9992              16.18
 enron                 mlp      0.9894     0.0009         0.9902          0.9884       0.9911   0.9897
            0.9992             117.39
```

## 6. Experiment E4 - generalisation across message types

The best classifier from E3 (by mean cross-validated F1) is trained on SMS only, e-mail only, and both corpora combined, and tested on both test sets.

**In [13] - notebook cell 20**

```
best = e3.groupby("model")["cv_f1_mean"].mean().idxmax()
print("Best model by mean CV F1:", best)
e4 = experiments.e4_generalisation(corpora, best)
e4.to_csv(experiments.RESULTS / "e4_generalisation.csv", index=False)
print(e4[["trained_on", "tested_on", "accuracy", "precision", "recall", "f1",
```

```
          "false_positive_rate"]].to_string(index=False))
```

Observed output

```
Best model by mean CV F1: linear_svm
  E4 train=sms      test=sms   F1=0.9653
  E4 train=sms      test=enron F1=0.4807
  E4 train=enron    test=sms   F1=0.2495
  E4 train=enron    test=enron F1=0.9913
  E4 train=combined test=sms   F1=0.8522
  E4 train=combined test=enron F1=0.9830
trained_on tested_on  accuracy  precision  recall     f1  false_positive_rate
       sms       sms    0.9913     0.9766  0.9542 0.9653               0.0033
       sms     enron    0.5444     0.5277  0.4414 0.4807               0.3614
     enron       sms    0.3140     0.1448  0.9008 0.2495               0.7710
     enron     enron    0.9916     0.9877  0.9948 0.9913               0.0113
  combined       sms    0.9585     0.7750  0.9466 0.8522               0.0398
  combined     enron    0.9839     0.9920  0.9742 0.9830               0.0072
```

## 7. Final system: model selection and held-out evaluation

For each channel (SMS, e-mail) the preprocessing level and classifier are selected by cross-validation on the training split only. The selected pipeline is refitted on the full training split, evaluated once on the test split, and saved to models/.

### In [14] - notebook cell 22

```
import train
sys.argv = ["train.py"]
t0 = time.perf_counter()
train.main()
print(f"Final training finished in {time.perf_counter() - t0:.0f} s")
```

Observed output

```
  sms   clean     naive_bayes          cv F1=0.9600
  sms   clean     linear_svm           cv F1=0.9588
  sms   clean     logistic_regression  cv F1=0.9564
  sms   stopwords naive_bayes          cv F1=0.9473
  sms   stopwords linear_svm           cv F1=0.9466
  sms   stopwords logistic_regression  cv F1=0.9480
  sms   stem      naive_bayes          cv F1=0.9422
  sms   stem      linear_svm           cv F1=0.9411
  sms   stem      logistic_regression  cv F1=0.9468
  sms   lemma     naive_bayes          cv F1=0.9402
  sms   lemma     linear_svm           cv F1=0.9448
  sms   lemma     logistic_regression  cv F1=0.9459
sms: selected {'mode': 'clean', 'model': 'naive_bayes', 'cv_f1': 0.96}
sms: test {'accuracy': 0.9894, 'precision': 0.9839, 'recall': 0.9313, 'f1': 0.9569, 'false_positive_rate':
    0.0022, 'tp': 122, 'fp': 2, 'tn': 902, 'fn': 9, 'roc_auc': 0.9963}
  email clean     naive_bayes          cv F1=0.9874
  email clean     linear_svm           cv F1=0.9908
  email clean     logistic_regression  cv F1=0.9895
  email stopwords naive_bayes          cv F1=0.9874
  email stopwords linear_svm           cv F1=0.9904
  email stopwords logistic_regression  cv F1=0.9894
  email stem      naive_bayes          cv F1=0.9871
  email stem      linear_svm           cv F1=0.9894
  email stem      logistic_regression  cv F1=0.9885
  email lemma     naive_bayes          cv F1=0.9869
  email lemma     linear_svm           cv F1=0.9899
  email lemma     logistic_regression  cv F1=0.9890
email: selected {'mode': 'clean', 'model': 'linear_svm', 'cv_f1': 0.9908}
email: test {'accuracy': 0.992, 'precision': 0.9887, 'recall': 0.9945, 'f1': 0.9916, 'false_positive_rate':
    0.0104, 'tp': 2895, 'fp': 33, 'tn': 3149, 'fn': 16, 'roc_auc': 0.9994}
saved models -> /home/claude/spam-detector/models
Final training finished in 591 s
```

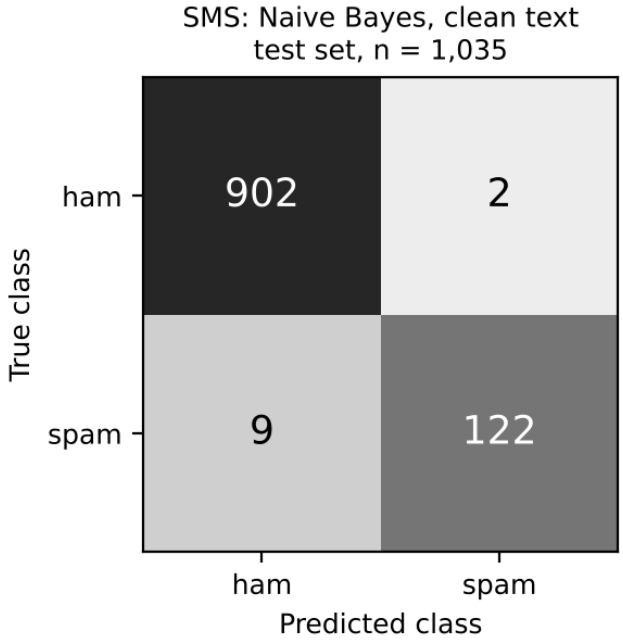
### In [15] - notebook cell 23

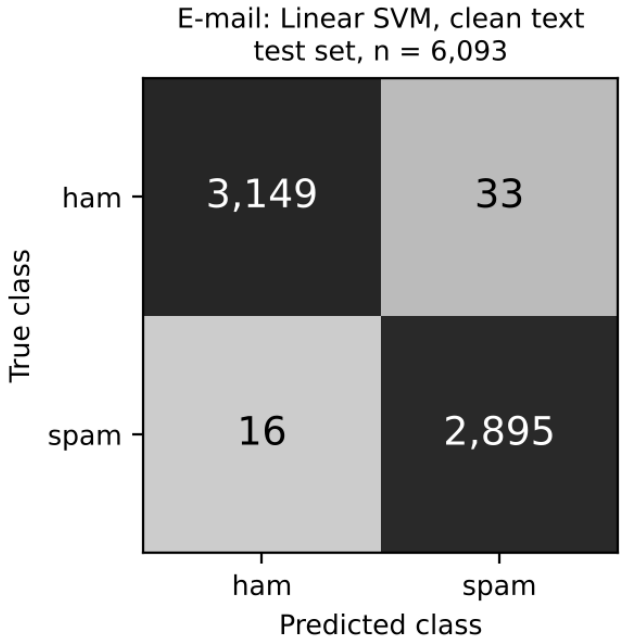
```
final = json.loads((ROOT / "results" / "final_model.json").read_text())
rows = []
for ch, r in final.items():
    rows.append({"channel": ch, "model": r["selected"]["model"], "preprocessing": r["selected"]["mode"],
                 "cv_f1": r["selected"]["cv_f1"], **{k: r["test"][k] for k in
                 ["accuracy", "precision", "recall", "f1", "roc_auc", "false_positive_rate",
                  "tp", "fp", "tn", "fn"]},
                 "predict_ms": r["predict_ms_per_message"]})
print(pd.DataFrame(rows).T.to_string(header=False))
for ch in final:
```

```
    display(Image(str(experiments.FIGURES / f"confusion_final_{ch}.png"), width=300))
```

Observed output

| channel | sms | email |
|---|---|---|
| model | naive_bayes | linear_svm |
| preprocessing | clean | clean |
| cv_f1 | 0.96 | 0.9908 |
| accuracy | 0.9894 | 0.992 |
| precision | 0.9839 | 0.9887 |
| recall | 0.9313 | 0.9945 |
| f1 | 0.9569 | 0.9916 |
| roc_auc | 0.9963 | 0.9994 |
| false_positive_rate | 0.0022 | 0.0104 |
| tp | 122 | 2895 |
| fp | 2 | 33 |
| tn | 902 | 3149 |
| fn | 9 | 16 |
| predict_ms | 0.047 | 0.859 |



## 8. Error analysis

Misclassified test messages of the final SMS model, and the n-grams that the final e-mail model weights most strongly towards each class.

### In [16] - notebook cell 25

```
import joblib
sms_model = joblib.load(ROOT / "models" / "sms_classifier.joblib")
_, sms_te = corpora["sms"]
pred = sms_model.predict(sms_te.text.tolist())
err = sms_te.assign(pred=pred)[pred != sms_te.label.to_numpy()]
# Sexually explicit spam texts are counted but not printed.
EXPLICIT = ("sex", "xxx", "babe", "bitch", "shit", "nasty", "rude", "cum", "hot")
def show(texts):
    clean = [t for t in texts if not any(w in t.lower() for w in EXPLICIT)]
    for t in clean: print("  -", t[:110])
    if len(clean) < len(texts): print(f"  ({len(texts) - len(clean)} explicit message(s) not printed)")
print(f"False positives (ham flagged as spam): {(err.label == 0).sum()}")
show(err[err.label == 0].text.tolist())
print(f"False negatives (spam missed): {(err.label == 1).sum()}")
show(err[err.label == 1].text.tolist())
```

Observed output

```
False positives (ham flagged as spam): 2
  - Nokia phone is lovly..
  - Unlimited texts. Limited minutes.
False negatives (spam missed): 9
  - Xmas & New Years Eve tickets are now on sale from the club, during the day from 10am till 8pm, and on
    Thurs, F
  - Missed call alert. These numbers called but left no message. 07008009200
  - In The Simpsons Movie released in July 2007 name the band that died at the start of the film? A-Green
    Day, B-B
  - Latest News! Police station toilet stolen, cops have nothing to go on!
  (5 explicit message(s) not printed)
```

### In [17] - notebook cell 26

```
email_model = joblib.load(ROOT / "models" / "email_classifier.joblib")
names = email_model.named_steps["vectorize"].get_feature_names_out()
coef = email_model.named_steps["classify"].coef_.ravel()
order = np.argsort(coef)
print("Strongest spam indicators:", ", ".join(names[order[::-1][:15]]))
print("Strongest ham indicators :", ", ".join(names[order[:15]]))

nb_ = sms_model.named_steps["classify"]; v = sms_model.named_steps["vectorize"].get_feature_names_out()
ratio = nb_.feature_log_prob_[1] - nb_.feature_log_prob_[0]
print("SMS model, highest spam/ham log-probability ratio:", ", ".join(v[np.argsort(ratio)[::-1][:15]]))
```

Observed output

```
Strongest spam indicators: your, http, remove, no, software, here, hello, you, re tok_num, sex, site, life,
    online, viagra, account
Strongest ham indicators : enron, vince, louise, thanks, attached, energy, doc, daren, i, gas, dave, enron
    com, employees, houston, the
SMS model, highest spam/ham log-probability ratio: to tok_phone, claim, prize, tok_phone, call tok_phone,
    tok_phone now, your mobile, nokia, have won, tok_num cash, tok_num prize, to claim, guaranteed,
    tok_phone to, tok_url
```

## 9. Classifying new messages

The saved pipelines accept raw text. Short texts are routed to the SMS model, long or multi-line texts to the e-mail model.

### In [18] - notebook cell 28

```
from predict import classify
new_messages = [
    "URGENT! Your mobile number has won a 2000 pound prize. Call 09061743810 to claim now",
    "hey, running 10 min late, can you order me a coffee?",
    "Are you free for a call about the budget tomorrow at 3?",
    "FREE entry! Win cash and a holiday, txt WIN to 80086 today",
    "Subject: lowest prices on meds\nBuy cheap medication online without prescription. "
    "Limited offer, click here to order now and save 80 percent.",
    "Subject: schedule for thursday\nHi Sally, please find attached the revised schedule "
    "for the pipeline meeting. Let me know if the volumes look right. Thanks",
]
for r in classify(new_messages):
    print(f"{r['label'].upper():4s} [{r['channel']:5s}] score={r['score']:+7.3f}  {r['message'][:70]!r}")
```

Observed output

```
SPAM [sms  ] score=+14.055  'URGENT! Your mobile number has won a 2000 pound prize. Call 0906174381'
HAM  [sms  ] score= -5.164  'hey, running 10 min late, can you order me a coffee?'
HAM  [sms  ] score= -3.156  'Are you free for a call about the budget tomorrow at 3?'
SPAM [sms  ] score= +7.256  'FREE entry! Win cash and a holiday, txt WIN to 80086 today'
SPAM [email] score= +1.908  'Subject: lowest prices on meds\nBuy cheap medication online without pre'
HAM  [email] score= -2.447  'Subject: schedule for thursday\nHi Sally, please find attached the revi'
```

## 10. Automated tests

### In [19] - notebook cell 30

```
import subprocess
out = subprocess.run([sys.executable, "-m", "pytest", "-q", str(ROOT / "tests")],
                     capture_output=True, text=True, cwd=ROOT)
print(out.stdout[-600:])
```

Observed output

```
......................                                                  [100%]
23 passed in 10.35s
```